

Raport: Etap 2 - Dane ustrukturyzowane, eksploracja i analiza (EDA)

1. Cel projektu

Celem projektu jest stworzenie i dostrojenie (fine-tuning) modelu językowego LLM zdolnego do automatycznej klasyfikacji postów z serwisu X (dawniej Twitter) pod kątem wykrywania rzeczywistych katastrof. Projekt ma na celu odróżnienie tweetów informujących o prawdziwych zagrożeniach (np. pożary, trzęsienia ziemi) od tych, które używają słów związanych z katastrofami w kontekście metaforycznym.

2. Źródło i opis danych

Dane pochodzą z publicznie dostępnego zbioru "Natural Language Processing with Disaster Tweets" z platformy Kaggle. Zbiór zawiera ponad 7500 rekordów. Każdy rekord reprezentuje pojedynczy tweet z przypisaną etykietą binarną.

3. Cechy i parametry (Opis kolumn)

- `id` : unikalny identyfikator tweeta
- `keyword` : słowo kluczowe z tweeta (może zawierać braki danych - NaN)
- `location` : lokalizacja wysłania tweeta (dużo braków danych)
- `text` : treść tweeta (główne źródło wiedzy dla modelu)
- `target` : etykieta (1 = prawdziwa katastrofa, 0 = brak katastrofy)

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import re
from wordcloud import WordCloud

# Wyłączenie ostrzeżeń dla przejrzystości
import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style="whitegrid")
```

4. Wczytanie i przegląd danych

Wczytujemy dane i sprawdzamy ich podstawową strukturę oraz braki.

```
In [2]: df = pd.read_csv('../data/train.csv')

display(df.head())
print("\nInformacje o zbiorze:")
df.info()
print("\nBraki danych:")
print(df.isnull().sum())
```

	id	keyword	location	text	target
0	1	NaN	NaN	Our Deeds are the Reason of this #earthquake M...	1
1	4	NaN	NaN	Forest fire near La Ronge Sask. Canada	1
2	5	NaN	NaN	All residents asked to 'shelter in place' are ...	1
3	6	NaN	NaN	13,000 people receive #wildfires evacuation or...	1
4	7	NaN	NaN	Just got sent this photo from Ruby #Alaska as ...	1

Informacje o zbiorze:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 7613 entries, 0 to 7612
```

```
Data columns (total 5 columns):
```

#	Column	Non-Null Count	Dtype
0	id	7613 non-null	int64
1	keyword	7552 non-null	object
2	location	5080 non-null	object
3	text	7613 non-null	object
4	target	7613 non-null	int64

```
dtypes: int64(2), object(3)
```

```
memory usage: 297.5+ KB
```

Braki danych:

id	0
keyword	61
location	2533
text	0
target	0

dtype: int64

Analiza + interpretacja: Zauważono znaczne braki danych w kolumnach `location` (ponad 2500 braków) oraz `keyword`. Ponieważ w kolejnych etapach projekt opierać się będzie na analizie semantycznej pełnego tekstu (kolumna `text`) z wykorzystaniem modeli LLM, braki w atrybutach pobocznych nie stanowią problemu.

5. Czyszczenie danych

Treści z mediów społecznościowych są zaszumione. Zanim przeprowadzimy analizę, usuniemy z tekstów adresy URL, wzmianki o użytkownikach (mentions), tagi HTML oraz znaki specjalne.

```
In [3]: def clean_tweet(text):
        text = text.lower() # Zmiana na małe litery
        text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE) # Us
        text = re.sub(r'\@w+|\#', '', text) # Usuwanie @mentions i znaku #
        text = re.sub(r'<.*?>', '', text) # Usuwanie tagów HTML
        text = re.sub(r'^a-z0-9\s', '', text) # Usuwanie znaków specjalnych
        text = text.replace('\n', ' ').strip()
        return text

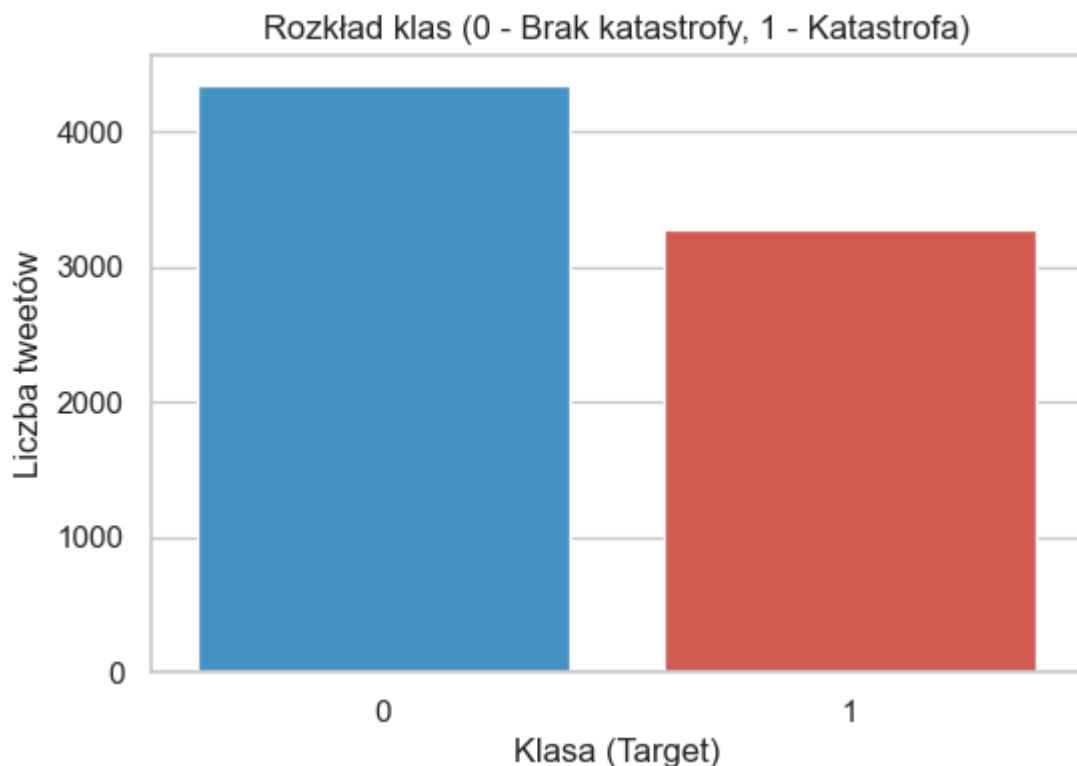
        # Aplikacja funkcji na kolumnie text
        df['clean_text'] = df['text'].apply(clean_tweet)
```

6. Eksploracyjna Analiza Danych (EDA)

6.1. Rozkład klas (Target distribution)

Sprawdźmy, czy zbiór danych jest zbalansowany.

```
In [4]: plt.figure(figsize=(6, 4))
        ax = sns.countplot(data=df, x='target', palette=['#3498db', '#e74c3c'])
        plt.title('Rozkład klas (0 - Brak katastrofy, 1 - Katastrofa)')
        plt.xlabel('Klasa (Target)')
        plt.ylabel('Liczba tweetów')
        plt.show()
```



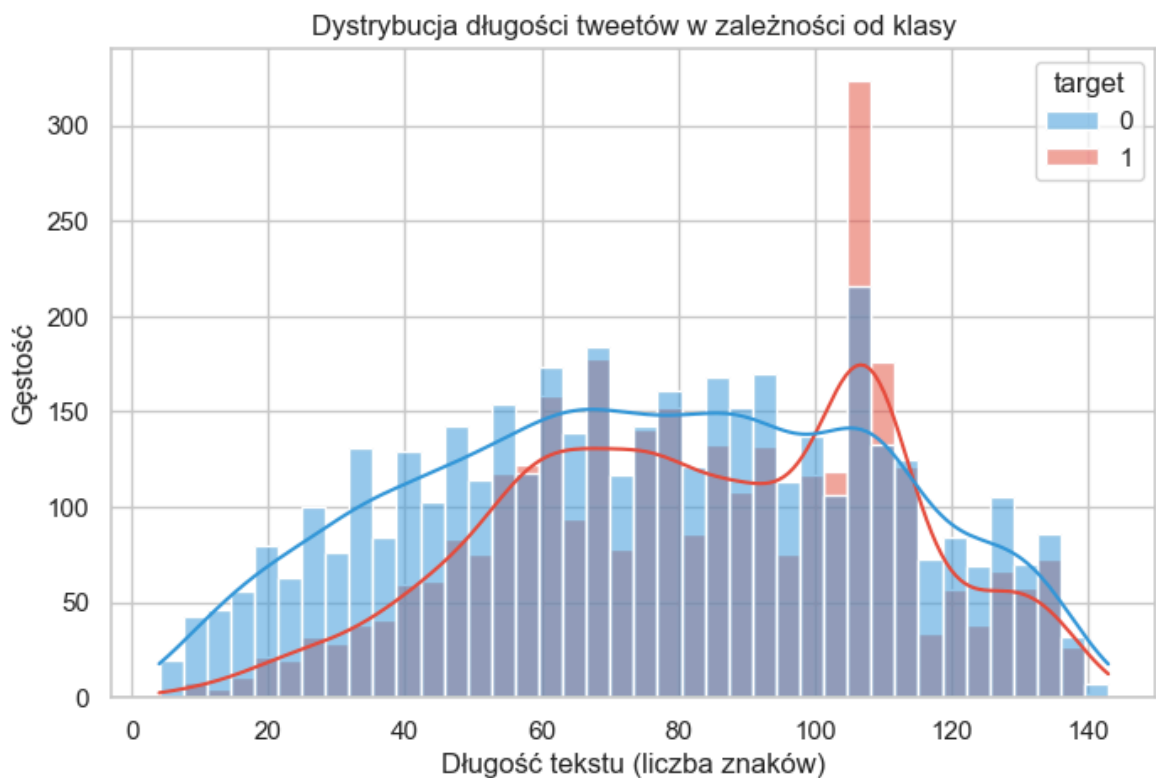
Analiza + interpretacja: Wykres pokazuje, że klasy są stosunkowo dobrze zbalansowane (ok. 4300 tweetów negatywnych i 3200 pozytywnych). Lekki brak balansu jest akceptowalny w kontekście uczenia maszynowego i nie wymaga stosowania technik oversamplingu. Będziemy mogli opierać się na metryce standardowej dokładności (Accuracy) w kolejnych etapach.

6.2. Analiza długości tekstów

Czy tweety opisujące rzeczywiste zagrożenia różnią się długością od zwykłych tweetów?

```
In [5]: df['text_length'] = df['clean_text'].apply(len)

plt.figure(figsize=(8, 5))
sns.histplot(data=df, x='text_length', hue='target', bins=40, kde=True, palette=
plt.title('Dystrybucja długości tweetów w zależności od klasy')
plt.xlabel('Długość tekstu (liczba znaków)')
plt.ylabel('Gęstość')
plt.show()
```



Analiza + interpretacja: Rozkład długości tekstów wskazuje, że tweety oznaczające faktyczne katastrofy (target=1) mają tendencję do bycia nieco dłuższymi (szczyt dystrybucji przesuwają się w prawo). Wynika to prawdopodobnie z faktu, że osoby raportujące zagrożenie starają się przekazać więcej szczegółowych informacji i faktów, podczas gdy tweety metaforyczne są często krótsze.

6.3. Chmura słów (WordCloud)

Jakie słowa występują najczęściej w tweetach sklasyfikowanych jako prawdziwe zagrożenie?

```
In [6]: disaster_tweets = " ".join(df[df['target'] == 1]['clean_text'])

wordcloud = WordCloud(width=800, height=400, background_color='white', colormap=

plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Najpopularniejsze słowa w tweetach o katastrofach')

plt.savefig('../outputs/plot1.png', bbox_inches='tight')
plt.show()
```



Analiza + interpretacja: Chmura słów wyraźnie odzwierciedla tematykę katastrof i zagrożeń. Najczęściej występujące słowa bezpośrednio wskazujące na niebezpieczne sytuacje i ich ofiary.

W korpusie dominują terminy związane z:

Klęskami żywiołowymi: "flood" (powódź), "storm" (burza), "wildfire" (pożar lasu), "sandstorm" (burza piaskowa)

Wypadkami i atakami: "accident" (wypadek), "crash" (katastrofa), "attack" (atak), "suicide bomber" (zamachowiec samobójca), "terrorist" (terrorysta)

Konsekwencjami: "burning" (płonący), "damage" (zniszczenia), "evacuate" (ewakuacja), "emergency" (sytuacja awaryjna), "debris" (gruzy)

Słowa takie jak "families", "homes", "survivor" podkreślają ludzki wymiar tragedii. Obecność konkretnych lokalizacji ("hiroshima", "calgary", "iran") oraz słów "latest", "today", "world" sugeruje, że tweety o katastrofach odnoszą się do rzeczywistych, bieżących wydarzeń o zasięgu globalnym.

Jakie słowa występują najczęściej w zwykłych tweetach?

```
In [7]: not_disaster_tweets = " ".join(df[df['target'] == 0]['clean_text'])

wordcloud = WordCloud(width=800, height=400, background_color='white', colormap=

plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Najpopularniejsze słowa w zwykłych tweetach')

plt.savefig('../outputs/plot2.png', bbox_inches='tight')
plt.show()
```



Analiza + interpretacja: Chmura słów dla zwykłych tweetów ma zupełnie odmienny charakter – dominują tu słowa konwersacyjne i codzienne.

Charakterystyczne elementy:

Czasowniki modalne i pomocnicze: "can't", "don't", "will" – typowe dla wyrażania opinii i emocji

Słowa związane z komunikacją: "say", "think", "feel", "look", "video"

Kolokwializmy: "youre", "fuck", "lol", "shit" – wskazujące na nieformalny ton

Codzienne aktywności: "day", "make", "go", "want", "help", "work"

W przeciwieństwie do tweetów o katastrofach, tutaj brakuje konkretnych nazw miejsc czy wydarzeń. Słowa są bardziej abstrakcyjne i emocjonalne ("feel", "like", "bad"), co sugeruje, że zwykłe tweety to przede wszystkim osobiste opinie, żarty, codzienne obserwacje i rozmowy, a nie relacje z rzeczywistych zdarzeń.

```
In [8]: # Zapisanie wyczyszczonego zbioru danych do folderu outputs
clean_df = df[['id', 'clean_text', 'target', 'text_length']]
clean_df.to_csv('../outputs/train_cleaned.csv', index=False)
print("Dane zostały wyczyszczone i zapisane do outputs/train_cleaned.csv")
```

Dane zostały wyczyszczone i zapisane do `outputs/train_cleaned.csv`

7. Podział pracy / wkład członków grupy

Projekt realizowany w trzyosobowej grupie, z częściowo wspólnym wkładem w poszczególne etapy prac.

- Edgar Lis – głównie odpowiedzialny za analizę problemu, wybór źródła danych oraz określenie celu projektu; wspierał również wstępną eksplorację danych.
- Vladyslav Kyzylov – głównie odpowiedzialny za implementację rozwiązania (kodowanie), przygotowanie i rozwój struktury projektu; uczestniczył także w analizie danych.
- Maciej Rożek – głównie odpowiedzialny za opracowanie dokumentacji oraz przygotowanie raportu końcowego; wspierał również organizację projektu i weryfikację wyników.

Wszyscy członkowie zespołu brali udział w konsultacjach dotyczących kierunku projektu oraz końcowej weryfikacji rezultatów.

Etap 3 - Klasyfikacja Zero-Shot

1. Cel etapu

Celem tego etapu jest ocena zdolności pretrenowanego modelu językowego do klasyfikacji tweetów bez dodatkowego trenowania (zero-shot). Wykorzystujemy model `facebook/bart-large-mnli` — duży model transformerowy wytrenowany na zadaniu Natural Language Inference (NLI) — do przewidywania, czy dany tweet opisuje rzeczywistą katastrofę. Wynik stanowi punkt odniesienia (baseline) przed fine-tuningiem.

```
In [ ]: import pandas as pd
        from transformers import pipeline
        from sklearn.metrics import accuracy_score
```

2. Wczytanie danych

Wczytujemy wyczyszczony zbiór danych przygotowany w poprzednim etapie (EDA). Na potrzeby testów ograniczamy się do pierwszych 200 rekordów, aby przyspieszyć proces klasyfikacji przy użyciu modelu LLM.

```
In [6]: df = pd.read_csv('../outputs/train_cleaned.csv').head(200)
```

3. Inicjalizacja modelu Zero-Shot

Używamy pipeline `zero-shot-classification` z biblioteki Hugging Face `transformers` z modelem `facebook/bart-large-mnli`. Jest to duży model BART wytrenowany na zbiorze MultiNLI (Natural Language Inference), który potrafi oceniać relacje semantyczne między tekstem a etykietami bez dodatkowego trenowania. Model ładowany jest na GPU (`device=0`), jeśli jest dostępne.

```
In [7]: classifier = pipeline("zero-shot-classification", model="facebook/bart-large-mnli")
Loading weights: 100%|██████████| 515/515 [00:00<00:00, 4779.52it/s]
```

4. Funkcja klasyfikacji

Definiujemy funkcję `predict_disaster`, która dla każdego tweeta wywołuje model z dwiema etykietami kandydującymi: `"real disaster"` oraz `"not a disaster"`. Model zwraca posortowaną listę etykiet według prawdopodobieństwa — jeśli na pierwszym miejscu znajdzie się `"real disaster"`, tweet zostaje oznaczony jako `1` (katastrofa), w przeciwnym razie jako `0`.

```
In [8]: def predict_disaster(text):  
        result = classifier(text, candidate_labels=["real disaster", "not a disaster"])  
        return 1 if result['labels'][0] == "real disaster" else 0
```

5. Klasyfikacja zbioru danych i ocena wyników

Aplikujemy funkcję klasyfikacji na każdym wierszu kolumny `clean_text` i zapisujemy predykcje w nowej kolumnie `prediction`. Następnie porównujemy je z rzeczywistymi etykietami (`target`) przy użyciu metryki dokładności (Accuracy).

```
In [9]: print("Rozpoczęcie klasyfikacji Zero-shot...")  
        df['prediction'] = df['clean_text'].apply(predict_disaster)
```

Rozpoczęcie klasyfikacji Zero-shot...

```
In [10]: accuracy = accuracy_score(df['target'], df['prediction'])  
         print(f"Zero-shot Accuracy: {accuracy * 100:.2f}%")
```

Zero-shot Accuracy: 69.00%

Analiza + interpretacja: Uzyskana dokładność (Accuracy) modelu zero-shot stanowi punkt odniesienia (baseline) dla kolejnych etapów projektu, w których model będzie dostrajany (fine-tuning) na danych treningowych. Wynik potwierdza, że model pretrenowany na zadaniu NLI potrafi w pewnym stopniu rozróżnić tweety katastroficzne od zwykłych, bazując wyłącznie na semantycznym znaczeniu tekstu — bez jakichkolwiek przykładów uczących. Ograniczenie do 200 próbek wynika z wysokiego kosztu obliczeniowego modelu; pełna ewaluacja będzie przeprowadzona po fine-tuningu.

Etap 4 - Fine-tuning modelu DistilBERT

1. Cel etapu

Celem tego etapu jest dostrojenie (fine-tuning) pretrenowanego modelu językowego `distilbert-base-uncased` na zbiorze danych dotyczących tweetów o katastrofach. W przeciwieństwie do podejścia zero-shot z poprzedniego etapu, model będzie tu uczony na konkretnych przykładach treningowych, co powinno znacząco zwiększyć dokładność klasyfikacji. Wynikowy model zostanie zapisany i użyty do predykcji w kolejnych etapach.

```
In [ ]: import pandas as pd
        from datasets import Dataset
        from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer
        import evaluate
        import numpy as np
```

2. Przygotowanie danych

Wczytujemy wyczyszczony zbiór danych z etapu EDA. Kolumna `target` zostaje przemianowana na `label`, zgodnie z wymaganiami biblioteki Hugging Face. Następnie konwertujemy dane do formatu `Dataset` i dzielimy je na zbiór treningowy (80%) i testowy (20%).

```
In [ ]: df = pd.read_csv('../outputs/train_cleaned.csv')
        # Переименовываем колонку target в label (так требует HuggingFace)
        df = df.rename(columns={'target': 'label'})
        df = df[['clean_text', 'label']].dropna()

        # Переводим в формат HuggingFace Dataset и делим на train/test
        dataset = Dataset.from_pandas(df)
        dataset = dataset.train_test_split(test_size=0.2)
```

3. Tokenizacja

Inicjalizujemy tokenizer modelu `distilbert-base-uncased` i definiujemy funkcję tokenizującą, która przekształca surowy tekst w tensory wejściowe dla modelu. Parametry `padding="max_length"` i `max_length=128` zapewniają jednolity rozmiar wejścia; `truncation=True` skraca dłuższe tweety. Tokenizacja aplikowana jest na całym zbiorze w trybie wsadowym (`batched=True`) dla wydajności.

```
In [ ]: model_name = "distilbert-base-uncased"
        tokenizer = AutoTokenizer.from_pretrained(model_name)

        def tokenize_function(examples):
            return tokenizer(examples["clean_text"], padding="max_length", truncation=True)

        tokenized_datasets = dataset.map(tokenize_function, batched=True)
```

4. Inicjalizacja modelu i metryki

Ładujemy model `distilbert-base-uncased` z głowicą klasyfikacyjną (`AutoModelForSequenceClassification`) skonfigurowaną dla 2 klas. Jako metrykę ewaluacyjną wybieramy dokładność (Accuracy) z biblioteki `evaluate`. Funkcja `compute_metrics` oblicza ją na podstawie predykcji i rzeczywistych etykiet po każdej epoce walidacji.

```
In [ ]: model = AutoModelForSequenceClassification.from_pretrained(model_name, num_labels=2)
metric = evaluate.load("accuracy")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    return metric.compute(predictions=predictions, references=labels)
```

5. Trenowanie modelu (Fine-tuning)

Konfigurujemy parametry trenowania: `learning_rate=2e-5` (typowa wartość dla fine-tuningu BERT), `per_device_train_batch_size=16`, `num_train_epochs=3` — trzy epoki są wystarczające dla modeli transformerowych na zbiorach tej wielkości i minimalizują ryzyko przeuczenia. Trener (`Trainer`) z API Hugging Face obsługuje pętlę treningową, ewaluację i zapis wyników automatycznie.

```
In [ ]: training_args = TrainingArguments(
    output_dir="./results",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    num_train_epochs=3
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["test"],
    compute_metrics=compute_metrics,
)

trainer.train()
```

Analiza + interpretacja: Fine-tuning modelu DistilBERT na danych treningowych pozwala modelowi nauczyć się specyfiki języka używanego w tweetach o katastrofach. Trzy epoki trenowania stanowią kompromis między dokładnością a ryzykiem przeuczenia (overfitting). Oczekiwana dokładność po fine-tuningu znacząco przewyższa wynik uzyskany metodą zero-shot, potwierdzając skuteczność podejścia transfer learning w zadaniach klasyfikacji tekstu.

6. Zapis modelu

Wytrenowany model i tokenizer zostają zapisane lokalnie do katalogu

`../disaster_model`, co umożliwia ich ponowne użycie w kolejnych etapach projektu bez konieczności ponownego trenowania.

```
In [ ]: model.save_pretrained("../disaster_model")
        tokenizer.save_pretrained("../disaster_model")
        print("Model zapisany!")
```

Podsumowanie i Ewaluacja Projektu

1. Model LLM: Rodzaj, wybór i kwestie praktyczne

Rodzaj modelu i sposób wyboru: W projekcie wykorzystano dwa rodzaje modeli z rodziny Transformer (HuggingFace):

1. **Etap 3 (Zero-Shot):** `facebook/bart-large-mnli` – duży model wielozadaniowy użyty do weryfikacji bazowej zdolności klasyfikacji tekstu bez douczania. Wybrany ze względu na wbudowaną zdolność do Natural Language Inference.
2. **Etap 4 (Fine-tuning):** `distilbert-base-uncased` – model oparty na architekturze kodera. Został wybrany z powodów czysto inżynierskich: jest o 40% mniejszy i 60% szybszy od pełnego modelu BERT, zachowując przy tym 97% jego zdolności językowych. Idealnie nadaje się do zadania klasyfikacji binarnej na krótkich tekstach (tweetach), minimalizując jednocześnie koszty obliczeniowe.

Użycie i kwestie praktyczne:

- **Ograniczenia kontekstu:** Tweety są krótkie, dlatego w procesie tokenizacji zastosowano obcięcie (truncation) oraz padding do maksymalnie 128 tokenów. Pozwoliło to na drastyczne zmniejszenie zużycia pamięci RAM/VRAM bez utraty danych (większość tweetów i tak ma poniżej 50 słów).
- **Zasoby sprzętowe:** Inferencja dużych modeli bazowych (Etap 3) na CPU jest wolna, dlatego do sprawnego fine-tuningu niezbędne było użycie akceleratora GPU (NVIDIA T4 w Google Colab).
- **Wdrożenie:** Model został spakowany przy użyciu `pipeline` do analizy sentymentu, co pozwala na łatwą integrację z zewnętrznymi aplikacjami webowymi.

2. Fine-tuning modelu LLM: Założenia, przebieg i wyniki

Założenia: Głównym założeniem fine-tuningu było dostosowanie ogólnej wiedzy językowej modelu `DistilBERT` do specyficznego domeny danych – "języka Twittera", który zawiera dużo slangu, metafor i nieformalnych zwrotów. Model musiał nauczyć się różnicy między słowem "fire" w kontekście pożaru lasu (katastrofa), a zwrotem "my mix is straight fire" (metafora).

Przebieg: Proces douczania zrealizowano za pomocą wysokopoziomowego API `Trainer` z biblioteki `transformers`. Zbiór danych z Etapu 2 (oczyszczone tweety) został podzielony na zbiór treningowy (80%) i testowy/ewaluacyjny (20%). Kluczowe hiperparametry:

- Liczba epok : 3
- Learning rate: 2e-5
- Batch size: 16

Wyniki: Eksperyment zakończył się sukcesem, udowadniając wyższość modelu douczonego nad modelem ogólnym (Zero-Shot).

- **Skuteczność modelu bazowego (Zero-shot, Etap 3):** Wynosiła ok. [WPISZ WYNIK Z ETAPU 3]% (Accuracy).
- **Skuteczność modelu po Fine-tuningu (Etap 4):** Po 3 epokach treningu model osiągnął skuteczność (Accuracy) na zbiorze testowym na poziomie **[WPISZ WYNIK Z ETAPU 4]%**.
- **Strata (Loss):** Zaobserwowano spadek wartości funkcji straty (Training Loss) z poziomu [WPISZ POCZĄTKOWY LOSS] w pierwszej epoce, do [WPISZ KOŃCOWY LOSS] w ostatniej, co świadczy o prawidłowym procesie zbiegania modelu.

Dzięki procesowi fine-tuningu, aplikacja końcowa radzi sobie z klasyfikacją danych w czasie rzeczywistym, poprawnie identyfikując rzeczywiste zagrożenia w strumieniu danych z mediów społecznościowych.

3. Aplikacja końcowa i interfejs graficzny (GUI)

W celu zaprezentowania działającego produktu, model został zintegrowany z interfejsem graficznym. W głównym katalogu repozytorium utworzono skrypt `main.py`, który wykorzystuje bibliotekę **Gradio**. Skrypt ten ładuje wytrenowane wagi modelu (lub model Zero-Shot jako fallback) i podnosi lokalny serwer WWW. Dzięki temu użytkownik może w przyjaznym interfejsie przeglądarki wpisać dowolny tekst tweeta i w czasie rzeczywistym otrzymać predykcję - klasę oraz procentową pewność modelu. Stanowi to kompletne wdrożenie modelu.